

Kapitel 5. Iteration

Innehållsförteckning

[while-slingor](#)[do-while-slingor](#)[for-slingor](#)[Avbryta och fortsätta slingor](#)

Detta kapitel behandlar de olika iterationsmöjligheterna i C++. Tre olika typer av slingor kan användas, alla med olika användningsområden. Med iteration avses en eller flera satser som upprepas noll eller flera gånger beroende på ett visst villkor.

while-slingor

Den enklaste av alla slingor i C++ är den så kallade `while`-slingan. Den används för att upprepa ett antal satser så länge som ett villkor håller. Den allmänna formen ser ut så här:

```
while ( villkor )  
    sats;
```

`while`-slingan upprepar `sats` så länge som `villkor` evalueras till `true`. Om villkoret aldrig är sant utförs slingan aldrig, utan exekveringen fortsätter direkt på raden efter. Villkoret evalueras efter varje exekvering av slingan. Exemplet nedan skriver ut talen från 1 till 10 på skärmen:

```
// skriver talen 1 till 10  
int Index = 1;  
while ( Index++ <= 10 )  
    cout << "Tal: " << Index << "\n";
```

Notera att värdet på `Index` ökas *efter* att `Index` använts, d.v.s. första iterationen har den värdet 1, och då iterationen är slut har den värdet 11. Vill man ha flera än en sats i slingans kropp bör man använda ett block av satser:

```
while ( villkor ) {  
    sats1;  
    sats2;  
    sats3;  
}
```

I detta fall utförs alla tre satser sekventiellt. Villkoret i `while`-slingan kan vara vad som helst som evaluerar till något som kan tolkas som `true` eller `false`. Det är rekommendabelt att alltid kapsla in `while`-slingans sats(er) i block, även om det endast är frågan om en enda sats. Det är lätt hänt att man senare vill utöka slingans innehåll, och om man då inte har satserna inom ett block händer det enkelt att man får något i stil med detta:

```
// beräknar en summa av talen 1 till 10  
int Index = 1;  
int Summa = 0;
```

```
while ( Index <= 10 )
    Summa = Summa + Index;
    Index++;
cout << "Summan är: " << Summa << "\n";
```

Programmet ser helt korrekt ut med en snabb titt, men ser man närmare på det märker man snabbt att endast raden `Summa = Summa + Index;` tillhör `while`-slingans kropp, och således är den enda rad som utförs. Raden `Index++` verkar enligt indenteringen att tillhöra slingan, men gör det inte. Resultatet blir en evig slinga och inget svar. Det korrekta förfarandet är att använda ett kodblock för slingans kropp:

```
// beräknar en summa av talen 1 till 10
int Index = 1;
int Summa = 0;
while ( Index <= 10 ) {
    Summa = Summa + Index;
    Index++;
}
cout << "Summan är: " << Summa << "\n";
```

Vi kommer att se på alternativa sätt att skriva slinga ovan senare i detta kapitel.

Eviga slingor

En *evig slinga* är en slinga som aldrig avslutas. Dylika kan vara praktiska då ett program skall köra en viss slinga tills det terminerar. Eftersom villkoret i en `while`-slinga är flexibelt kan en evig slinga skrivas t.ex. på följande sätt:

```
while ( true ) {
    cout << "Hello world!\n";
}
```

I exemplet ovan kan slingans villkor aldrig evalueras till `false`. Vi kunde även skrivit villkoret som `1` eller ett annat tal som inte är `0`, eftersom de evalueras till `true`.

Nästlade slingor

Ibland kan det vara praktiskt att kunna nästla slingor. Detta kan förstås göras även med de slingor som finns i C++. Man kan ha flera slingor innanför varandra, med det enda kravet att de måste vara helt innanför varandra, de kan inte överlappa på något sätt. Olika typer av slingor kan förstås även nästlas. Exemplet nedan skriver ut talen 1 till 100 i en fyrkant, med 10 tal per rad:

```
int Rad = 0;
int Kolumn = 1;
while ( Rad < 100 ) {
    while ( Kolumn < 10 ) {
        // skriv ut ett tal och inkrementera kolumnen
        cout << Rad + Kolumn;
        Kolumn++;
    }

    // en rad full, gå till nästa rad
    cout << "\n";
    Rad = Rad + 10;
}
```

Vi kommer senare i detta kapitel att se ett elegantare sätt att utföra samma sak, men med hjälp av `for`-slingor (se [avsnittet `for-slingor`](#)).

[Företgående](#)Headerfilen `iostream`[Hem](#)[Nästa](#)`do-while-slingor`